Experiment-10

Write a python program to implement A* algorithm

Aim

To write and execute a Python program to find the shortest path from the start node S to the goal node D using the A* algorithm with the evaluation function f(n) = g(n) + h(n).

Algorithm

1. Represent the graph with edge costs and give a heuristic value h(n) for each node.
2. Insert the start node S into the open list (priority queue) with f(S) = h(S).
3. Remove the node with the lowest f(n) value from the open list.
4. If the removed node is the goal node, stop and display the path.
5. Otherwise, expand the node and calculate g(n) and f(n) = g(n) + h(n) for each neighbour.
6. Insert the neighbours into the open list.
7. Store the expanded nodes in the closed list to avoid repetition.
8. Repeat the process until the goal node is reached.
9. Display the shortest path and the total cost.

Coding

ex-10.py - D:/ex-10.py (3.10.0)

File Edit Format Run Options Window Help

```
# A* Algorithm   f(n) = g(n) + h(n)
import heapq

# Graph with edge costs
graph = {
    "S": [("A", 1), ("B", 4)],
    "A": [("B", 2), ("C", 5), ("D", 12)],
    "B": [("C", 2)],
    "C": [("D", 3)],
    "D": []
}

# Heuristic value (estimated cost to reach the goal D)
h = {"S": 7, "A": 6, "B": 4, "C": 2, "D": 0}

def a_star(start, goal):
    open_list = [(h[start], 0, start, [start])]
    closed = set()

    while open_list:
        f, g, node, path = heapq.heappop(open_list)
        print("Expanding", node, " g =", g, " h =", h[node], " f =", f)

        if node == goal:
            return path, g

        if node in closed:
            continue
        closed.add(node)
```

ex-10.py - D:/ex-10.py (3.10.0)

File Edit Format Run Options Window Help

```
        for neighbour, cost in graph[node]:
            new_g = g + cost
            new_f = new_g + h[neighbour]
            heapq.heappush(open_list,
                           (new_f, new_g, neighbour, path + [neighbour]))
    return None, 0

print("A* ALGORITHM")
print("------------")
path, cost = a_star("S", "D")
print()
print("Shortest Path :", " -> ".join(path))
print("Total Cost    :", cost)
```

Output

IDLE Shell 3.10.0

File Edit Shell Debug Options Window Help

```
Python 3.10.0 (tags/v3.10.0:b494f59, Oct  4 2021, 19:00:18) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========================== RESTART: D:/ex-10.py ==========================
A* ALGORITHM
------------
Expanding S  g = 0  h = 7  f = 7
Expanding A  g = 1  h = 6  f = 7
Expanding B  g = 3  h = 4  f = 7
Expanding C  g = 5  h = 2  f = 7
Expanding B  g = 4  h = 4  f = 8
Expanding C  g = 6  h = 2  f = 8
Expanding D  g = 8  h = 0  f = 8

Shortest Path : S -> A -> B -> C -> D
Total Cost    : 8
>>>
```

Result

The A* algorithm was successfully implemented. The program found the shortest path:

S -> A -> B -> C -> D

with a total cost of 8.